---

title: Build Work Estimate — Copper Basin Utilities -- outage triage agent author: Dewain Robinson build_platform: github-copilot target_platform: copilot-studio estimate_id: est_20260904T000000_sample3 generated: 2026-09-04T00:00:00Z

Build Work Estimate — Copper Basin Utilities -- outage triage agent

Author: Dewain Robinson · **Estimate id:** `est_20260904T000000_sample3` · **Generated:** 2026-09-04T00:00:00Z

> **SAMPLE — BUILD ESTIMATE ONLY, NOT A QUOTE**
>
> This document is generated by a **sample estimator** published as a demonstration of *how* an organization could build one. It is not a quote, a bid, a budget authority, or a commitment of any kind.
>
> **It estimates the work of building only — never the cost of running what gets built.** Runtime, end-user licences, infrastructure, and human labour are all outside its scope and are not represented in any figure below.
>
> **The figures are estimates and will be wrong.** They are derived from historical consumption patterns that may not resemble the work being estimated. Observed cost for comparable work in the calibration data spans more than 100x between the cheapest and most expensive instances. Treat the range as the estimate; treat the point figure as the midpoint of a guess.
>
> **Before any real budgeting use, this estimator must be modified for your organization** — recalibrated against your own usage history, repriced against your contracted rates rather than list price, and adjusted for your own delivery patterns, model choices, and review overhead.
>
> Rates verified 2026-09-03 and change without notice. Calibration source: `measured` · Generated 2026-09-04T00:00:00Z

Estimate summary

What was estimated

INPUT	VALUE
Built with (development tool)	GitHub Copilot — metered in GitHub AI Credits
Built on (target environment)	Microsoft Copilot Studio — metered in Copilot Credits
Built by (model)	gpt-5.4 50% / gpt-5.5 25% / gpt-5-mini 25%
Target harness	`standard` — does not bill for build or test
Licensing	Seat — GitHub Copilot Business, $19.00/month x 4 seats x 3 months = **$228.00** over the build
Specification	`in-review` — confidence in sizing: **medium**
Evaluation cycles planned	2
Contingency reserve	30%
Work sizing	Turn counts from measured Claude Code calibration (24 sessions), used as a work-size proxy

Totals

	BUILD — GITHUB AI CREDITS	TARGET — COPILOT CREDITS	COMBINED (USD)
Low	1,089	0	$10.89
Likely	**4,350**	**0**	**$43.50**
High	24,159	0	$241.59
Likely + 30% reserve	**5,655**	**0**	**$56.55**

> **Plan for $43.50. Hold $56.55 including the 30% reserve.**

Two meters, not two options. The build and target figures are spent on the same project over the same period and add together; they are not alternatives to choose between.

- The target column is **zero because of the harness**, not because nothing was estimated. On the standard harness, build, preview, test and evaluation in the interface are not billed, and agent flow test runs are explicitly exempt. See the target section below.
- The build figure is **notional** — on seat licensing no additional money is invoiced. See Licensing below for the share of the seat this build actually consumes.
- The build side is sized from the same work breakdown using turn medians measured on Claude Code. Turn count is treated as a property of the work rather than of the tool; the price per turn is not. That is an assumption, not a measurement of GitHub Copilot.

Everything below explains how each of these figures was reached.

What this was sized from

	REFERENCE
Functional specification	Outage intake and triage requirements (illustrative)
Technical specification	Copilot Studio agent design note (illustrative)
Status	`in-review` — Under review; scope may still move.
Confidence in sizing	**medium**

Platforms

	PLATFORM	METERED IN
Built with	GitHub Copilot	GitHub AI Credits
Built on	Microsoft Copilot Studio	Copilot Credits

> These are different questions and different meters, and the same project spends on **both**. The building happens in a coding agent authoring the agent definition; the target platform is where it is deployed, previewed, evaluated and validated. Copilot Studio is a destination, not a build tool.
>
> **This is not a platform comparison tool.** It reports one chosen build platform and one chosen target, each in its own meter. Platform decisions are not made on cost alone — capability, existing skills, governance, integration, and support all matter more than a build-time figure — and using this report to pick a platform would be using it for something it was not designed to answer.

Scope — this estimates the build, not the run

This figure covers the work of **building** the thing. It does not cover operating it afterwards.

OUT OF SCOPE	WHERE TO GO INSTEAD
Runtime / operational cost of what gets built	Copilot Studio agent usage estimator
End-user seat licences (M365 Copilot, Power Platform)	Your licensing/procurement process
Infrastructure, hosting, storage, egress	Your cloud cost tooling
Human labour — PM, design, QA time	Your delivery estimation process
Ongoing maintenance and support after delivery	A run cost, not a build cost

The build loop

```
 GitHub Copilot                Microsoft Copilot Studio
-----------------------        -----------------------
author definition   -->  deploy
                         preview / interactive test  <- human
                         run evaluations
remediate           <--  evaluations fail
(repeat)
```

This estimate plans **2 evaluation cycles**. Each cycle after the first adds **25%** of the original build back as remediation, giving a build-side multiplier of **1.25x**. An estimate that prices only the first pass is planning for a build where every evaluation passes first time.

Environments

> **No environments were declared.** This estimate prices the solution as if it were built once, into one place. A real delivery provisions dev, QA, test and production — each needing infrastructure applied, pipelines wired, configuration and secrets set, data seeded, and the agent deployed.

Declaring them is the single largest correction available to this estimate.

Build cost — GitHub Copilot

> Metered in **GitHub AI Credits**. This is a different meter from Copilot Studio Copilot Credits, even though both are $0.01 per credit.

Billing model: Usage-based GitHub AI Credits — Interactions consume input, output, and cached tokens. GitHub prices those at the model's published rates and converts the result to AI Credits at $0.01 per credit. On Business and Enterprise plans credits are pooled at the billing-entity level.

Build model: `gpt-5.4 50% / gpt-5.5 25% / gpt-5-mini 25%` — rates published GitHub rates, blended across 3 models.

GitHub prices AI Credits from token consumption at the selected model's published rate, so which model builds is a cost input rather than a preference. Rate verified 2026-09-04 against models and pricing.

BUILD ACTIVITY	GITHUB AI CREDITS	BASIS
Chat and agent interactions	4,350	1,789 interactions, 10,734,000 tokens (15% output) at $2.56/$15.50 per 1M, less 10% Auto discount
Total	**4,350**	
Reserve (30%)	1,305	required contingency
Budget ask	**5,655**	

At $0.01 per credit that is **$43.50** total, **$56.55** with reserve.

Not metered

These consume no credits and are unlimited on paid plans, so they contribute nothing to this estimate however heavily they are used:

- Code completions
- Next edit suggestions

Rates verified 2026-09-03. Sources: models and pricing · legacy premium requests · plans

Target platform — Microsoft Copilot Studio

Target harness: `standard` — Standard harness bills after publish, and embedded test chat messages are not billed. Build-time credits are therefore near zero. Non-zero only where the build exercises billable side-effects

(agent flow runs, AI Builder or content-processing calls) against a published agent.

> **On the standard harness, build and test in the interface are not billed.** Billing starts after publish and embedded test chat does not count, so the work below consumes no credits. That is a correct result, not a missing one.

Had this been the GitHub Copilot harness, the same volume of preview and evaluation work would have cost roughly **3,960 credits ($39.60)** — worth knowing before choosing a harness.

The evaluation loop

Evaluation is not a one-off gate. Microsoft's guidance is an explicit cycle — define tests, run evaluations, analyse results, improve the agent, repeat — with a target pass rate of 80–90% and near 100% on core tests.

This estimate plans **2 cycles**: 240 evaluation runs in total, plus the remediation each failed cycle sends back to the build platform.

> **Velocity, not just cost.** Evaluations are capped at **20 per agent node per day**, so this volume needs a minimum of **12 days** of elapsed time however much budget is available.

Human validation — a dependency, not a cost line

12.0 hours of interactive validation are planned in the Copilot Studio interface. Those hours are collected to size the test volume above — they are **not** estimated as labour cost, consistent with this tool metering agent consumption rather than people.

Someone must go into the interface, confirm behaviour, and make configuration changes between cycles. That step gates the loop, and no amount of build budget removes it.

Not included

- Production runtime once the agent is live
- Agent flow test runs in the designer or test chat (documented exemption)
- Capacity pack sizing and overage enforcement
- End-user Microsoft 365 Copilot licence offsets
- Human hours for validation (collected to size test volume only)

For production runtime consumption use Microsoft's agent usage estimator.

Rates verified 2026-09-03. Sources: billing rates · harness billing · evaluation limits · iteration guidance

What is measured, and what is judgment

This estimate mixes two kinds of input. They are not equally trustworthy, and the difference is not visible in the figures themselves.

Measured or sourced

INPUT	BASIS
Cost per agent turn, bucket turn medians, cache behaviour	Derived from 24 real local sessions
Every provider rate	Published, each carrying a source URL and a verification date
Evaluation volume	Arithmetic on declared test cases, repeats and cycles
Azure consumption	Figures supplied by you; nothing is bundled or inferred

Judgment, not measurement

These shape the result and **were never measured against anything**. They are stated here so a reader can see which parts of the number would move if someone measured them.

FACTOR	VALUE	WHAT IT DOES
Brownfield factor	**1.50x**	Multiplies turns for work in an existing codebase. Judgment. Never measured against paired greenfield and brownfield work.
Remediation share per cycle	**25%**	Each evaluation cycle after the first adds this share of the build back as rework. Judgment. Real remediation depends on what the evaluations actually find.
Unknowns range widening	**+25% per unknown**	Widens the upper bound of an item per declared unknown. Judgment. The declared unknown count is itself a subjective input.
Environment provisioning share	**25% per extra environment**	Cost of applying infrastructure and pipeline work into each environment beyond the first. Judgment. Override it with `environments.provisioning_share` if you have a real figure.
Evaluation cycle range	**-1 / +2 cycles**	Produces the low and high bounds on the target side. Judgment. Nobody knows how many cycles a build will need until it runs.
Correction shrinkage k	**k = 3**	Pulls recorded actuals toward 1.0 so one data point cannot swing later estimates. Judgment, but a deliberately conservative one: it only ever reduces the influence of thin data.

> **Nothing here is invented at report time.** Every figure above is either measured, supplied by you, or one of the listed factors applied to those. Where the estimator cannot attribute a cost honestly — target credits with no declared evaluation cases, for instance — it says so rather than distributing the total to make the table look complete.

Provenance of every figure

Every money figure and every grouped number above has been checked against the estimator's own derivation ledger. Each one is either measured from session history, a published rate carrying a source URL and verification date, a value you declared in the manifest, or arithmetic on those.

Nothing in this report is asserted without a derivation. The check is mechanical and runs on every build; a figure the estimator cannot account for fails validation rather than being printed.

Matching is exact at the precision each figure is rendered to. There is **no tolerance window**, so a value one away from a recorded one fails.

What it does **not** verify: that a recorded value appears in the right place. A renderer showing a real figure under the wrong label would pass. That is a different defect needing a different check, and claiming otherwise would be the overstatement this section exists to avoid.

Known limits

1. **Build only.** Nothing here is the cost of running the agent once it is live.
2. **Human validation is a dependency, not a line item.** Hours are collected to size test volume, never estimated as labour.
3. **Turn counts are the weakest input**, and thin buckets are flagged above.
4. **Rates go stale.** Re-verify against the sources cited.
5. **This is a sample.** Modify it for your organization before budgeting use.

Close the loop

```
python record_actual.py est_20260904T000000_sample3 --sessions <session-id> [...]
```

SAMPLE — build estimate only, not a quote. Excludes runtime cost. Modify for your organization before budgeting use.